Project and Build

CVM++ is a programming language and interpreter shipped as a single program called `cvmpp`. Source code lives in `.cvm` files. The interpreter reads a file, turns it into bytecode, and runs that bytecode on a small stack virtual machine. Everything is implemented in C++17 without parser generators or LLVM.

Pipeline and runtime

End-to-end flow

When we run `cvmpp examples/hello.cvm`:

1. The **driver** (`main.cpp`) reads the file from disk.
2. `compile_frontend()` in `compile.cpp` runs the **lexer**, **parser**, and **compiler**.
3. The result is a `BytecodeChunk` — a byte array plus metadata (global names, function table).
4. Unless we pass `-c` (compile only), the **virtual machine** in `vm.cpp` executes those bytes.
5. Output goes to stdout; errors print with a phase name and a caret under the source line.

The AST exists only while compiling. At runtime the VM never sees the tree — only the bytecode.

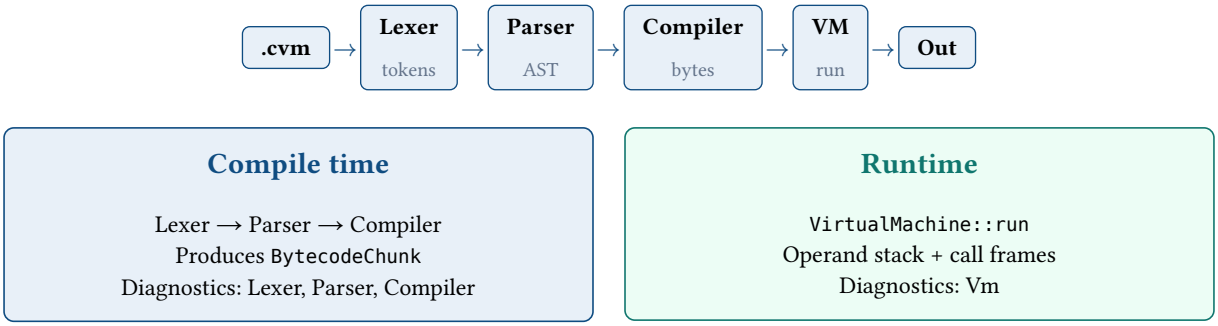

Figure 1: Compile time (front end) versus runtime (virtual machine).

Stage	Input	Output	Source files
Lexer	characters	`vector<Token>`	`lexer.cpp`, `token.cpp`
Parser	tokens	`Program` (AST)	`parser.cpp`, `ast.hpp`
Compiler	AST	`BytecodeChunk`	`compiler.cpp`, `bytecode.cpp`
VM	bytecode	values / errors	`vm.cpp`, `value.cpp`

CLI modes and debug

File mode. With a path argument, `main` reads the file, compiles once, and runs it.

REPL. With no arguments, each line is compiled and executed in a `VmSession` that keeps globals between lines (`let x = 1` on one line is visible on the next). Commands: `:help`, `:disasm path`, `:quit`.

Flags.

Flag	Effect
`-d`	Print tokens, AST, and bytecode — then run
`-c`	Compile only; no VM
`-q`	Quiet run (no debug banners)

Using `-d` to localize bugs.

1. **Tokens** — confirms the lexer sees keywords and operators.
2. **AST** — confirms parse structure (`ast_print.cpp`).
3. **Bytecode** — confirms jump targets and literals (`disassemble()`).

Wrong tokens ⟶ lexer. Wrong AST ⟶ parser. Wrong run with good AST ⟶ compare bytecode to section 5 and `vm.cpp`.

Scope and design

Goals

1. One repository and one binary: lex, parse, compile, execute.
2. Language subset: integers, booleans, variables, `fn`, `if`/`while`, `print`, `input`, recursion.
3. Diagnostics tagged by phase (`Lexer`, `Parser`, `Compiler`, `Vm`, `Repl`) with source location.
4. Eleven `.cvm` examples and `make verify` (CI runs the same checks).

Scope

In v1	Not in v1
Hand-written lexer, parser, compiler, stack VM	Strings, floats, arrays
REPL with persistent globals	Modules / imports
`fn`, `CALL`, `RETURN`, jump patching	`for`, `break`, `continue`
Entry `JUMP` at offset 0 ⟶ main	Separate compiler / VM binaries
`-d`, `-c`, disassembler	Yacc/Lex, LLVM
11 tests + `verify.sh` + CI	Large standard library

Design choices

Area	Decision	Reason
Types	`int64` and `bool`	Simple VM checks
AST	`unique_ptr` tree	Clear ownership, one compile pass
Bytecode	`vector<uint8_t>` + name pool	Compact, easy to disassemble
Functions	`FunctionMeta` + `CALL` by index	Address and arity at compile time
Layout	`JUMP` at byte 0 ⟶ main	`fn` bodies emitted before main code
REPL	Global map in `VmSession`	Variables persist across lines
Safety	Stack 65536; 1M steps per run	Overflow and infinite-loop guards

Build phases

Each phase ended with something runnable from the terminal.

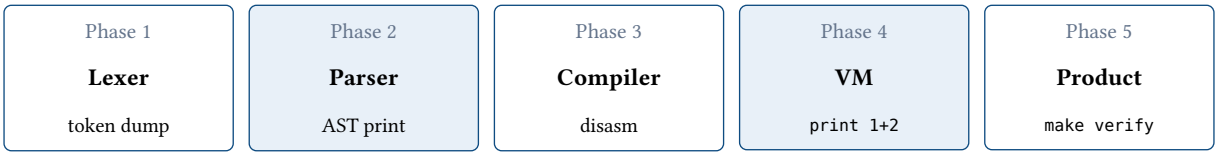

Figure 2: Implementation order: each phase unlocks the next and has a concrete verification step.

Phase	Focus	Check
1	Lexer + diagnostics	`let x = 42;` ⟶ six tokens
2	Parser + AST	`cvmpp -d hello.cvm` ⟶ syntax tree
3	Compiler	`cvmpp -c` ⟶ bytecode listing
4	Virtual machine	`print 1 + 2;` ⟶ `3`

5	Functions, REPL, release	`make verify` → 11 passed

Phase 1 — Lexer and diagnostics

The lexer turns text into tokens (type, lexeme, source location). `SourceLoc`, `Diagnostic`, and `DiagnosticBag` tag errors with `Phase::Lexer`. `Lexer::tokenize()` handles keywords, `//` comments, multi-char operators, and integer overflow. `ui.cpp` prints errors with a caret.

What we built `source_loc`, `diagnostic`, `lexer`, `token`, `ui`. For `let x = 42;`: `Let`, `Identifier`, `Assign`, `Integer`, `Semicolon`, `Eof`.

How we checked Token dump for a small input file.

Phase 2 — Parser and AST

The parser builds a `Program` tree. `ast_print` wired to `-d` separates parse bugs from compiler bugs.

What we built `Expr` / `Stmt` in `ast.hpp`. `Program` has `functions[]` and `statements[]`. Recursive-descent precedence. `synchronize()` recovers after errors.

How we checked `./build/cvmpp -d examples/hello.cvm` — tokens then indented AST.

Phase 3 — Bytecode compiler

The compiler walks the AST into `BytecodeChunk::code`. Control flow becomes jump instructions with patched offsets. This phase used the disassembler; the VM came in phase 4.

What we built `opcode.hpp`, `BytecodeWriter` (`emit`, `patch_u32`, `intern_name`). `compile_if` / `compile_while` emit placeholders then patch. `fn` / `CALL` added in phase 5.

How we checked `cvmpp -c examples/arithmetic.cvm` — bytecode only. `-d` also runs the VM.

Phase 4 — Virtual machine

The VM dispatch loop runs until `Halt` or error. First end-to-end target: `print 1 + 2;` → `3`.

What we built `VmValue` (`int` / `bool`). Binary ops pop **right** then **left**. Uninitialized globals are errors. `compile_frontend()` does not execute; `main` calls `execute()`. Guards: ÷0, stack, types, step limit.

How we checked Disasm shows `PushInt 1`, `PushInt 2`, `Add`, `Print`. Run prints `3`, exit `0`.

Phase 5 — Functions, REPL, and release

User functions, recursion, `!=`/`<=`/`>=`, REPL, examples, CI. Fixed bytecode layout bug when `fn` bodies sit at low addresses.

What we built `CALL` / `RETURN`, locals, `Ne`/`Le`/`Ge`, `VmSession`, flags `-d`/`-q`/`-c`, eleven `.cvm` files, `scripts/verify.sh`, GitHub Actions.

How we checked `make verify` — 11 passed. `functions.cvm` prints `120`.

Bytecode layout and entry jump

The VM starts at offset 0. Top-level `fn` bodies are emitted **before** main (lower addresses). Without a skip, byte 0 would enter a function with no call frame → `LOAD_LOCAL outside function`.

`compile_program()` (`compiler.cpp`):

1. Emit `JUMP` + placeholder at the start.
2. Compile all functions.
3. Patch `JUMP` to main entry.

4. Compile main statements.
5. Emit `Halt`.

Bytecode layout

Order in `code[]`	What lives there
Start	`JUMP` → skip to main
Middle	All `fn` bodies (low addresses)
After patch	Main script statements
End	`HALT`

Language reference

Syntax

Feature	Form	Notes
Literals	`42`, `true`, `false`	64-bit integers and booleans
Variable	`let x = expr;` then `x = expr;`	Globals in files; REPL persists
Function	`fn name(a, b) { ... }`	Top-level; `return expr;`
Call	`name(arg1, arg2);`	Arity checked at compile time
If / else	`if (cond) { ... } else { ... }`	`cond` must be `bool`
While	`while (cond) { ... }`	Condition re-checked each iteration
Print	`print expr;`	Value + newline
Input	`let x = input;`	One stdin line as integer

Operator precedence

Tightest operators bind first:

Level (tight → loose)	Operators	Example
Unary	`-expr`	`-5`
Multiply / divide	`* /`	`8 / 2` → `4`
Add / subtract	`+ -`	`1 + 2 * 3` → `7`
Comparison	`< > <= >=`	`x < 10`
Equality	`== !=`	`a == b`

Control flow and calls

If / else: condition → `JumpIfFalse` (placeholder) → `then` → optional `Jump` over `else` → patch targets.

1. Compile condition expression → stack
2. `JUMP_IF_FALSE` → placeholder offset (else / exit)
3. Compile `then` branch
4. Optional `JUMP` over `else`; patch false target
5. Compile `else`; patch forward jump targets

Figure 3: `if` lowering: condition compiled first; false branch jumps around `then` or to `else`.

While: loop start → condition → `JumpIfFalse` to exit → body → `Jump` back → patch exit.

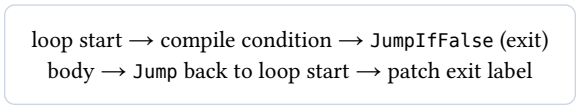

Figure 1: `while` lowering: condition at top of loop; exit jump patched after the body.

Call: arguments on stack ⟶ `Call` (index, arity) ⟶ callee locals ⟶ `Return` to caller.

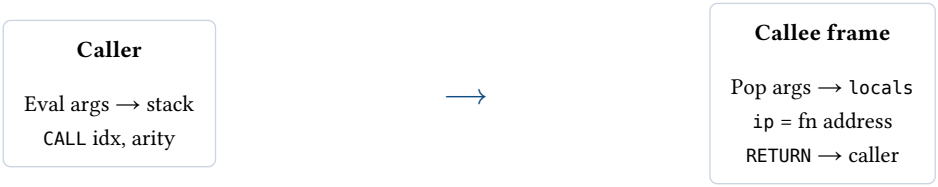

Figure 4: Function call: arguments on operand stack become callee local slots.

Abstract syntax tree

`Program` = `functions[]` + `statements[]` (main, after entry jump). `cvmpp -d` prints:

```
Program
|-- Let (x)
|   +-- IntLiteral (42)
|-- Let (flag)
|   +-- BoolLiteral (true)
|-- Print
|   +-- Variable (x)
+-- If
    |-- Binary (<)
    |   |-- Variable (x)
    |   +-- IntLiteral (100)
    +-- Block
        +-- Print
            +-- Binary (+)
                |-- Variable (x)
                +-- IntLiteral (8)
```

Expression nodes: literals, `Variable`, `Unary`, `Binary`, `Call`, `Input`.

Statement nodes: `Let`, `Assign`, `Print`, `Return`, `If`, `While`, `Block`, `ExprStmt` (emits `Pop`).

Opcodes

One opcode byte + fixed operands (`opcode.hpp`). Binary ops pop right, then left.

Opcode	Hex	Operands	Meaning
`PushInt`	0x01	i64	Integer literal
`PushBool`	0x02	u8	Boolean
`Pop`	0x03	—	Discard stack top
`LoadVar` / `StoreVar`	0x10 / 0x11	u16	Global
`LoadLocal` / `StoreLocal`	0x12 / 0x13	u8	Local in frame
`Add`–`Div`	0x20–0x23	—	Integer math; `/` checks ÷0
`Eq` `Ne` `Lt` `Gt` `Neg` `Le` `Ge`	0x24–0x2A	—	Compare / negate
`Input` / `Print`	0x30 / 0x31	—	Stdin line / print + newline
`Jump` / `JumpIfFalse`	0x40 / 0x41	u32	Branch
`Call` / `Return`	0x42 / 0x43	u16, u8 / —	Function call / return
`Halt`	0xFF	—	Stop

Example programs

Scripts live in `examples/`. `make verify` checks every exit code (ten scripts ⟶ `0`, `div_by_zero.cvm` ⟶ 2).

Script	Exit	Topic
`hello.cvm`	0	Variables, `print`, `if`
`arithmetic.cvm`	0	Operators
`if_else.cvm`	0	`else`
`factorial.cvm`	0	`while`
`functions.cvm`	0	Recursion, `CALL`
`input_demo.cvm`	0	`input`
`div_by_zero.cvm`	2	Runtime error
`booleans.cvm`	0	Booleans
`assignment.cvm`	0	Reassignment
`comparisons.cvm`	0	`!=`, `<=`, `>=`
`multiline_demo.cvm`	0	Multi-line blocks

Worked examples

Variables, print, and if

`examples/#file`

Bind globals, print, and branch on a condition.

Source

```
let x = 42;
let flag = true;
print x;
if (x < 100) {
  print x + 8;
}
```

Explanation

1. `let x = 42;` — global `x` = `42` (`StoreVar` after pushing the literal).
2. `let flag = true;` — boolean global (unused later; checks bool parsing).
3. `print x;` ⟶ first line `42`.
4. `if (x < 100)` — `42 < 100` is `true`; then-branch runs.
5. `print x + 8;` ⟶ `42 + 8` = `50` on the second line.

Run `./build/cvmpp -q examples/hello.cvm`

Output

```
42
50
```

Exit code 0

Arithmetic and comparisons

`examples/#file`

`a = 20`, `b = 6`; each `print` applies one opcode to the stack.

Source

```
let a = 20;
let b = 6;
print a + b;
print a - b;
print a * b;
```

```
print a / b;
print a == 20;
print a < b;
```

Explanation

1. `a + b` ⟶ `26` (`Add` pops `6`, then `20`).
2. `a - b` ⟶ `14` (right operand popped first).
3. `a * b` ⟶ `120`; `a / b` ⟶ `3` (integer division).
4. `a == 20` ⟶ `true`; `a < b` ⟶ `false`.

Run `./build/cvmpp -q examples/arithmetic.cvm`
Output

```
26
14
120
3
true
false
```

Exit code 0

If / else branches

`examples/#file`

`n = 17` so `n < 10` is false — only the else branch runs.

Source

```
let n = 17;
if (n < 10) {
  print 1;
} else {
  print 2;
}
```

Explanation

1. `JumpIfFalse` skips the then-block when the condition is `false`.
2. `print 2` in the else branch ⟶ output `2`.

Run `./build/cvmpp -q examples/if_else.cvm`
Output

```
2
```

Exit code 0

While loop (factorial)

`examples/#file`

Iterative `5!` with a `while` loop.

Source

```
let n = 5;
let result = 1;
while (n > 0) {
  result = result * n;
  n = n - 1;
```

```
}
print result;
```

Explanation

1. Loop: while `n > 0`, multiply `result` by `n`, then decrement `n`.
2. When `n` reaches `0`, exit and print `result`.

Run `./build/cvmpp -q examples/factorial.cvm`

Output

```
120
```

Exit code 0

Iteration	`n` after	`result`
1	4	5
2	3	20
3	2	60
4	1	120
5	0	120

Recursive functions

`examples/#file`

Recursive `factorial`; function bytecode before main; entry `JUMP` at offset 0.

Source

```
fn factorial(n) {
  if (n <= 1) {
    return 1;
  }
  return n * factorial(n - 1);
}

let x = factorial(5);
print x;
```

Explanation

1. `factorial(5)` ⟶ `Call` pushes a frame; `n` is local slot 0.
2. Base case `n <= 1` ⟶ `return 1`.
3. Else `n * factorial(n - 1)` — inner call completes before multiply.
4. Chain `5→4→…→1`, then multiply back ⟶ `120`.

Run `./build/cvmpp -q examples/functions.cvm`

Output

```
120
```

Exit code 0

```
Program
|-- Function (factorial)
|   |-- Param (n)
|   +-- Block ...
|-- Let (x)
|   +-- Call (factorial)
|       +-- IntLiteral (5)
+-- Print
    +-- Variable (x)
```

Input from stdin

examples/#file

Read stdin, print double.

Source

```
let n = input;
print n * 2;
```

Explanation

1. `let n = input;` reads one line as an integer.
2. `print n * 2` — with `echo 21 | cvmpp ...`, output is 42.

Run `echo 21 | ./build/cvmpp -q examples/input_demo.cvm`

Output

```
42
```

Exit code 0

Divide by zero

examples/#file

Compiles; VM errors at `Div` with divisor zero.

Source

```
let a = 10;
let b = 0;
print a / b;
```

Explanation

1. `Div` sees `10` and `0` on the stack.
2. VM prints `Phase::Vm` error; exit `2` (compile errors use `1`).
3. `make verify` requires exit `2` for this file.

Run `./build/cvmpp -q examples/div_by_zero.cvm`

Output

```
VM error at 0:0: divide by zero
hint: check the divisor before division; 0 is not allowed
```

Exit code 2

Repository and commands

Layout

Headers: `include/cvm++/`. Implementation: `src/`. One binary links everything.

Part	Role	Files
Diagnostics	Errors with carets	`source_loc`, `diagnostic`, `ui`
Lexer	Tokenize	`lexer`, `token`
Parser	Build AST	`parser`, `ast`, `ast_print`
Compiler	AST → bytecode	`compiler`, `bytecode`, `opcode`
VM	Execute	`vm`, `value`, `compile`
Driver	CLI, REPL	`main`, `runtime_options`
Tests	Regression	`examples/`, `scripts/verify.sh`
CI	Automated verify	`.github/workflows/ci.yml`

Build and verify

Command	Effect
`make`	Build `build/cvmpp` (`clang++`, `-Iinclude`)
`make cmake`	CMake Release build
`make verify`	Run `scripts/verify.sh` on all examples
`make clean`	Remove `build/cvmpp`

`scripts/verify.sh` checks stdout and exit codes. CI runs `make verify` on push.

Exit codes and VM guards

Code	When
0	Success
1	Lexer, parser, or compiler error
2	VM runtime guard (÷0, bad type, unread variable, …)

VM faults (exit 2): division by zero; stack overflow/underflow (max 65536); unread global; type mismatch; invalid `RETURN` / `LOAD_LOCAL`; more than 1M instructions per run.

Daily commands

Command	Use
`make`	Build
`make verify`	Run all examples
`cvmpp -d file.cvm`	Debug dump, then run
`cvmpp -c file.cvm`	Compile only
`cvmpp -q file.cvm`	Quiet run
`cvmpp`	REPL